

Ariel Henry

Agentic AI Section 003

February 4th, 2026

1. Project Proposal: AI Academic Success Interactive System

Universities struggle to proactively identify and support students who are at risk of academic failure, delayed graduation, or withdrawal. Academic advisors often manage hundreds of students and rely on limited indicators such as final grades or manual check-ins. By the time performance issues are visible, it may already be too late for effective intervention.

TARGET USERS: Undergraduate students, Academic advisors, University administrators

WHY IS AN AGENTIC AI SOLUTION APPROPRIATE HERE: An agentic system adds efficiency to this system because a planner agent can determine what metrics to analyze, a data agent can compute performance indicators, a risk agent can assess the probability of failure, an intervention agent can generate personalized strategies, and a critic agent can validate recommendations for consistency.

2. USE CASES & SCENARIOS

Scenario 1: Student checks to see if they're at risk of failing in a course and how to improve.

User wants to: "A student wants to know if they are at risk of failing CIS3001 and what to do this week to improve."

- **Intake Agent** collects grade data, attendance, and assignment completion.
- **Data Analysis Agent** calculates projected final grade and performance trends.
- **Risk Agent** assigns a risk level (Low/Moderate/High).
- **Intervention Agent** generates a short-term improvement plan.
- **Report Agent** produces a clear, student-friendly summary.

Scenario 2: Advisor triages any students and prioritizes outreach.

User wants to: “An academic advisor wants to scan 800 students and identify who needs help urgently, plus why.”

- **Data Agent** processes student performance data (CSV upload).
- **Risk Prediction Agent** assigns risk scores to each student.
- **Visualization Agent** generates charts showing risk distribution.
- **Summary Agent** creates a prioritized outreach list with recommended actions.

Scenario 3: Student plans next semester to stay on track for graduation and what courses to take next.

User wants to: “A student wants a recommended schedule for next semester that keeps them on track and balances difficulty.”

- **Requirement Analyzer Agent** compares completed credits to degree requirements.

- **Planning Agent** generates optimized semester schedules.
- **Risk Agent** evaluates course load difficulty.
- **Report Agent** provides a clear graduation roadmap

3. INITIAL SYSTEM DESIGN

Preliminary Agents & Roles

Intake/UI Agent: Collects student or advisor inputs & converts them into structured data.

Planner Agent: Selects which analyses to run (risk check, triage, risk indicators, credit progress).

Risk Assessment Agent: Generates risk scores & identifies key drivers of performance issues.

Intervention Agent: Suggests actions such as study plans, tutoring, or outreach to advisors.

Visualization Agent: Produces simple charts for trends and risk levels.

Critic/Auditor Agent: Validates logic, checks for unrealistic advice, and ensures outputs match computed data.

Report/Summarizer Agent: Creates student-friendly summaries or advisor-ready reports.

Expected Tools / API's / Data Sources

Opal Agents to coordinate Planner, Analyzer, Summarizer, and Critic roles.

Google Gemini API for reasoning, summaries, and recommendations.

Google Sheets / Drive APIs for storing mock student data and generating reports.

Local CSV/Excel files for synthetic datasets.

Simple UI built with Streamlit or Google App Sheet.

4. FEASIBILITY & RISKS

Potential Challenges

Hallucinations: The LLM may generate incorrect explanations, risk scores, or recommendations if not grounded in the provided data.

Long Context Limits: Large student histories or multi-step analyses may exceed context windows, requiring summarization or chunking.

Tool/Agent Failures: External API calls (Sheets, Drive, Search) may fail or return incomplete data, which can affect downstream agents.

Inconsistent Outputs: Multi-agent workflows may produce conflicting results without a strong Critic/Auditor agent.

Constraints

Privacy & Ethics: Only synthetic or anonymized student data will be used; no real grades or personal information.

Rate Limits: Gemini and Google APIs may limit request volume, requiring batching or caching.

Computation: All processing must remain lightweight enough to run within Opal and standard Google tooling.

Scope: Features must stay simple enough to build and test within the course timeline.

Project Timeline

The project will be completed **within the duration of the course**, which meets on Thursdays from 9:30 AM to 12:00 PM. Development will also include additional work outside of class hours to ensure steady progress and timely completion. The scope remains focused on delivering a functional prototype before the end of the term.